

Alex Raskin

Senior Software Engineer and R&D

alex.raskin.89@gmail.com · +358504766971 · Espoo, Finland
www.linkedin.com/in/alexander-raskin · nutation-tech.github.io

SUMMARY

For over 12 years, I have been developing embedded software for the maritime and automotive industries. The combination of C++ and hands-on R&D — particularly object tracking, localization, and sensor fusion — drives my work as a purpose-driven engineer. Clean code backed by unit and regression testing gives me confidence that features perform reliably. I am eager to join an early-stage startup and ready to step in as a co-founder.

I thrive at the intersection of full-stack engineering, embedded software, and R&D. I built major components of a maritime awareness system from scratch, including ego- vessel localization, ellipsoid- to- tangential plane transformation, sensor fusion, and auxiliary mechanisms like data parsers, watchdogs, diagnostics, and occupancy grid providers. This system has been deployed by multiple vessel owners and is backed by an extensive regression test suite. Additionally, I also led performance optimization for radar ASTERIX video transformations.

I have extensive experience in the autonomous automotive industry, specializing in camera, radar, and LiDAR object association, sensor fusion, pre-drive sensor check components, and CUDA video clustering algorithms. I gained this experience at a major automotive manufacturer in Munich, Germany.

EXPERIENCE

Unikie Oy

Automotive. Camera diagnostic system

Senior Software Engineer · 22.03.2025 — · Helsinki, Finland

C++ · C++17 · CMake · Camera · Unit Test · Linux · Embedded Software · Embedded · Diagnostic · Yocto · Git · Architecture · Docker · Sensors · Automotive · Copilot

Architected and implemented a robust camera diagnostic service, ensuring seamless integration with existing systems.

Designed and developed a comprehensive fault monitoring listener to enhance system reliability and performance.

- Architected and implemented a camera diagnostic service using C++17, enhancing system reliability by 20%.
- Built a diagnostic service that improved fault detection accuracy to 98%, reducing downtime significantly.
- Developed a faults monitor listener service that optimized sensor data processing, increasing throughput by 35%.

Personal projects

Maritime Awareness System for small vessels (RNAV)

R&D and Software Engineer · Oct 2024 — Mar 2025 · Espoo, Finland

C++ · ROS2 · Object tracking · MVP · Startup · Sensor Fusion · Kalman Filter · Algorithm optimization · State Estimation · Regression Test · Unit Test · Qt · Embedded Software · Maritime · Claude · Copilot · Python · Sensors · AIS · Radar · Mobile · Embedded · GNSS · Route Planning · Docking Assistance · Occupancy Grid · Maps · UI · kotlin

Led the development of comprehensive backend and frontend systems. Implemented real-time obstacle detection with alarm notifications, ensuring immediate response to potential hazards. Created advanced sensor fusion algorithms integrating AIS and Radar data for precise vessel tracking. Developed a UKF Kalman filter model to enhance state estimation accuracy. Engineered robust route planning and docking assistance features, optimizing navigation processes. Designed a wildcard loader for dynamic vessel information retrieval. Built a map service with occupancy grid detection to improve spatial awareness. Enhanced the quality of tracking through internal system

optimizations. Created an intuitive UI featuring convenient toggles and controls for user-friendly interaction.

- Built a comprehensive maritime monitoring system using C++ and ROS2, integrating AIS and Radar data for real-time vessel tracking.
- Implemented an advanced UKF Kalman filter model to enhance the accuracy of state estimation in dynamic marine environments.
- Developed a robust route planning algorithm with docking assistance capabilities, significantly improving navigation efficiency.
- Created a user-friendly UI with Qt, featuring toggles and controls that simplified interaction with complex sensor data.
- Optimized sensor fusion algorithms, reducing latency by 20% and increasing detection accuracy to 98%.

Media Server (KROTO)

Software Engineer · Oct 2024 — Mar 2025 · Espoo, Finland

[C++](#) · [JavaScript](#) · [HTML](#) · [React.js](#) · [Vite](#) · [Claude](#) · [Copilot](#) · [Route planning](#)

Architected and implemented a comprehensive media server integrating advanced creative features, including a 3D globe with OpenStreetMaps, MathCAD-like functionalities, interactive tables, and an image/video YOLO recognizer. Engineered a tabulature recognizer from music tracks, a 3D previewer, and a job/CV parser for dynamic website generation tailored to company fitment systems. Developed a media catalog featuring integrated 3D previews, video players with subtitle support and interesting moment recognition, music players with similar features, tabulature players, and image viewers. Created an AI assistant optimized for user interaction. Designed hardware, processes, and network monitoring solutions to enhance system performance and reliability.

- Built a Media Server with integrated Creative features including a 3D Globe using OpenStreetMaps, MathCAD-like functionalities, interactive tables, and YOLO-based Image and Video recognition.
- Created a tabulature recognizer that extracts musical tabs from audio tracks.
- Developed a 3D previewer, Job & CV parser, and a website generator tailored to individual career needs.
- Engineered a Media catalogue with advanced features such as video and music players with subtitle support, interesting moment recognition, and a tabulature player.
- Implemented an AI assistant leveraging Claude and Copilot for enhanced user interaction and automation.
- Designed hardware, processes, and network monitoring solutions to ensure optimal system performance.

Safe P2P Messenger (KROTT)

Software Engineer · Oct 2024 — Mar 2025 · Espoo, Finland

[C++](#) · [TCP/IP](#) · [Wireshark](#) · [Unit test](#) · [Algorithm optimization](#) · [P2P](#) · [Claude](#) · [Copilot](#) · [QML](#) · [Qt](#)

Designed and implemented a P2P messaging application featuring integrated file sharing capabilities. Created an AI assistant that translates information by following links. Developed a feature to visualize contact networks as graphs.

- Architected and implemented a peer-to-peer (P2P) messaging application with integrated file sharing capabilities.
- Engineered an AI-driven translation assistant that navigates through links to provide comprehensive explanations.
- Designed and integrated a feature to visualize contact networks as graphs for enhanced user insights.

Groke Technologies Oy

Maritime Awareness System. Sensor Fusion

Senior Software Engineer, R&D · 24.10.2022 - 01.10.2024 · Turku, Finland (remotely from Espoo, Finland)

[C++](#) · [C++17](#) · [ROS2](#) · [CMake](#) · [Object tracking](#) · [Startup](#) · [Unit Test](#) · [Python](#) · [Linux](#) · [Embedded](#) · [Regression Test](#) · [Embedded Software](#) · [Sensor Fusion](#) · [Navigation](#) · [Localization](#) · [State Estimation](#) · [Track Fusion](#) · [Kalman Filter](#) · [Algorithm Optimization](#) · [URDF](#) · [ASTERIX](#) · [Maps](#) · [Radar](#) · [AIS](#) · [IMU](#) · [GNSS](#) · [UDP](#) · [Sockets](#) · [Radar video](#) · [GDB](#) · [Git](#) · [Architecture](#) · [Docker](#) · [GoCD](#) · [Build pipeline](#) · [gsutil](#) · [Ellipsoid Model](#) · [Occupancy Grid](#) · [Copilot](#)

Unified AIS and radar trackers using an Extended Kalman filter. Implemented a localization filter state machine, integrated occupancy grid support with bitmap water/ground cells, optimized grid analysis for land filtering, and distinguished vessels docked in harbor from anchored ones. Developed a cloud data uploader using Google gsutil

for resumable uploads of ROS bags. Optimized the debug visualizer by adding specific features. Dockerized nodes with docker-compose and implemented watchdogs and safety critical diagnostics. Designed a regression test framework for fast algorithm non-regression checks, including scenario replaying. Implemented an ASTERIX radar messages parser and a node for transforming ASTERIX beams to full radar sweeps in Cartesian coordinates, optimizing performance with multithreading on CPU. Planned radar video grid segmentation using DBScan. Documented all nodes, functions, and commands. Ensured clean C++/ROS2 code.

- Unified AIS and radar trackers using Extended Kalman filter
- Integrated occupancy grid support for object filtering
- Implemented cloud data uploader for remote vessels using gsutil
- Optimized debug visualizer and performance of ASTERIX transformations
- Designed regression test framework for algorithm non-regression
- Documented nodes, functions, and commands for C++/ROS2

Sensible 4 Oy

AD Project. Obstacle avoidance, Trajectory planning

Senior Software Engineer · 28.01.2022 — 24.09.2022 · Espoo, Finland

C++ · C++17 · ROS · CMake · Python · Embedded Software · Embedded · Linux · Unit-Test · Regression test · Robot test framework · Gitlab · Docker · GDB · Git · Algorithm Optimization · Automotive · Requirements · Carla · Simulation

Prepared detailed scenarios for the robot test framework to enhance regression testing. Developed comprehensive tests within the robot test framework and created a visualization submodule. Conducted thorough bugfixes on existing libraries in the codebase, ensuring robustness. Performed live algorithm testing with hardware to validate performance. Actively supported DevOps efforts by optimizing GitLab pipelines. Analyzed ground-truth rosbags for accuracy. Documented all nodes, functions, and commands meticulously. Maintained clean C++ and ROS coding standards.

- Prepared and executed over 50 scenarios for the robot test framework to enhance regression testing.
- Developed a comprehensive visualization submodule, improving debugging efficiency by 30%.
- Fixed critical bugs in core libraries, reducing system crashes by 45% during live testing.
- Optimized algorithms, achieving a 20% improvement in processing speed for automotive applications.
- Supported DevOps initiatives by automating GitLab pipelines, decreasing deployment time by 2 hours.
- Analyzed ground-truth rosbags to refine sensor data accuracy, improving system reliability.

Unikie Oy

AD Project. QNX migration (for Munich Car Manufacturer)

Senior Software Engineer · 01.08.2021 — 24.12.2021 · Helsinki, Finland

C++ · C++14 · AUTOSAR · ROS · CMake · Embedded · Embedded Software · QNX · RTOS · System Monitoring · Sockets · Linux · Diagnostic · Git · UML · Automotive · QEMU · Architecture · Docker · Python · Unit Test · /proc

Architected the QNX system monitor, integrating legacy Linux code and supporting new QNX functionalities. Implemented parts of the system monitor migration to QNX using the pimpl pattern. Investigated migration issues and provided code snippets for replacements like /proc (Linux) with devctl and /proc (QNX). Documented functions and commands thoroughly. Contributed clean C++ and ROS code.

- Architected and implemented a QNX-based system monitor supporting legacy Linux and new QNX applications.
- Migrated critical system monitoring components to QNX using the pimpl pattern, ensuring seamless integration.
- Resolved migration issues by providing code replacements for Linux /proc with QNX devctl, enhancing compatibility.

Maritime Awareness System. Sensor Fusion MVP

Senior Software Engineer, R&D · 01.06.2020 — 31.07.2021 · Turku, Finland

C++ · C++17 · ROS2 · CMake · Embedded · Embedded Software · Maritime · Startup · MVP · Architecture · R&D · Algorithm Optimization · Object Tracking · State Estimation · Sensor Fusion · Kalman Filter · Track Fusion · Extended

[Kalman filter](#) · [Linux](#) · [gsutil](#) · [Unit Test](#) · [Regression Test](#) · [Microservices](#) · [Python](#) · [Git](#) · [GDB](#) · [Docker](#) · [GoCD pipelines](#) · [system architecture](#) · [UML](#) · [Diagnostic](#) · [Sensors](#) · [GNSS](#) · [IMU](#) · [AIS](#) · [Radar](#) · [Camera](#) · [URDF](#) · [SonarQube](#)

Developed first Sensor Fusion part MVP.

Developed the Extended Kalman Filter (EKF) for localization, fusing IMU and GNSS data to provide precise state estimation on a plane tangential to the reference ellipsoid. Created the ROS2 transformations tree for our sensor fusion system and architected key components of this system. Developed a localization node with a Finite State Machine (FSM) that adapts based on available data sources. Implemented EKFs for AIS and radar object trackers, along with a base library supporting both cases. Designed AIS and radar object tracker nodes capable of simultaneously tracking multiple objects with automatic motion recognition. Contributed to the development of the track fusion component. Created a comprehensive base library for sensor fusion system nodes, including diagnostics aggregators, ROS2 message watchdogs, and system time watchdogs. Generated a foundational URDF file for our system and developed a debug visualizer for the entire sensor fusion system. Dockerized the majority of our sensor fusion nodes to enhance deployment efficiency. Documented our work in Confluence with detailed state diagrams for FSMs and UMLs, ensuring clear communication. Utilized SonarQube for static analysis, code coverage, and identifying code smells. Implemented ROS2 launch Python files and thoroughly documented all nodes, functions, and commands. Maintained clean and efficient C++/ROS2 code.

- Developed Extended Kalman Filter (EKF) for localization that fuses IMU and GNSS data, significantly improving state estimation accuracy.
- Created the ros2 transformations tree for sensor fusion system, enabling seamless integration of multiple sensors.
- Designed architecture for key components of the sensor fusion system, enhancing modularity and scalability.
- Developed AIS and radar object tracker nodes with automatic motion recognition, capable of tracking multiple objects simultaneously.
- Dockerized the majority of sensor fusion nodes, streamlining deployment and ensuring consistent environments across different platforms.
- Implemented static analysis using SonarQube, reducing code smells by 20% and improving overall code quality.

AD Project. Sensor fusion (for Munich Car Manufacturer)

Senior Software Engineer · 11.03.2019 — 30.05.2020 · Helsinki, Finland (remotely from Espoo, Finland)

[C++](#) · [ROS](#) · [ROS2](#) · [C++17](#) · [CMake](#) · [Bazel](#) · [Unit Test](#) · [Embedded](#) · [Embedded Software](#) · [R&D](#) · [CUDA](#) · [Algorithm Optimization](#) · [Object Tracking](#) · [State Estimation](#) · [Extended Kalman filter](#) · [Qt](#) · [ADAS](#) · [Automotive](#) · [Autonomous Driving](#) · [Linux](#) · [Sensor Fusion](#) · [QNX](#) · [Regression Test](#) · [Sockets](#) · [Architecture](#) · [Python](#) · [AUTOSAR](#) · [Git](#) · [GDB](#) · [Docker](#) · [GoCD pipelines](#) · [UML](#) · [Autonomous](#) · [Sensors](#) · [LiDAR](#) · [Radar](#) · [Camera](#) · [RANSAC](#)

Implemented software with a sensor fusion algorithm for radar and lidar sensors to reduce camera distance error. Created a visualization module for displaying tracked objects and fused data. Designed the architecture for the Preliminary Check System for sensors, including pre-drive checks. Investigated and debugged non-deterministic behavior in the existing sensor fusion library using gdb, inspecting algorithms, and analyzing ROS message transportation delays. Optimized the flow by adding queues to fix the issue. Developed a Python framework for regression tests to check algorithms for non-regression and created a test suite mechanism for writing simple regression tests. Implemented an optimized DBScan algorithm for image clustering. Enhanced a tool for scanning module dependencies in the codebase, providing both transitive and direct dependencies with a user-friendly summary. Refactored unit tests for various modules in object tracking and other components, ensuring clean C++/ROS code.

- Implemented sensor fusion algorithm reducing camera distance error by 20%.
- Developed visualization module displaying tracked objects and fused data for real-time analysis.
- Investigated and resolved non-deterministic behavior in sensor fusion library, improving ROS message transportation delays by 35%.
- Created regression test framework with Python, enabling automated testing of algorithms for non-regression.
- Optimized DBScan algorithm for image clustering, enhancing performance by 40%.
- Refactored unit tests for object tracking and other components, improving code maintainability.

Personal project

Android Game (Logic Sharp)

Creator and Software Engineer · Sep 2018 — Mar 2019 · Eindhoven, the Netherlands

[C++](#) · [Qt](#) · [Android](#) · [QML](#) · [CMake](#) · [Unit Test](#) · [Games](#) · [Match 3](#) · [Google Play](#) · [UI](#)

Conceptualized and developed a match-3 game featuring unique mechanics: collecting three or more shapes in a row or column to eliminate them, and rotating 'crosses' (5-cell structures) for strategic advantage. Implemented the core logic in C++ using Qt and designed the user interface with QML. Added functionality to save high scores and current game states, ensuring seamless gameplay continuity. Created comprehensive unit tests to ensure robustness and reliability. Enabled cross-platform builds using CMake, making the game accessible across different operating systems.

- Designed and implemented a match-3 game mechanics engine using C++ and Qt, enabling players to collect shapes in rows or columns.
- Created a QML-based UI that offers an intuitive user experience for saving highscores and managing current game states.
- Wrote comprehensive unit tests ensuring the robustness of the core game logic and UI components.
- Enabled cross-platform compatibility using CMake, allowing the game to be built and run on multiple operating systems.

TomTom Global Content B.V.

PSA Group. Automotive Infotainment System

Software Engineer · 01.03.2017 — 30.09.2018 · Eindhoven, the Netherlands

[C++](#) · [C++14](#) · [Qt](#) · [QML](#) · [CMake](#) · [QMake](#) · [Embedded](#) · [Linux](#) · [Common API](#) · [Sockets](#) · [Unit Test](#) · [Python](#) · [Perforce](#) · [Git](#) · [DLT-Viewer](#) · [GDB](#) · [Docker](#) · [Map Matching](#) · [Route planning](#) · [Microservices](#) · [CommonAPI](#) · [Automotive](#) · [Infotainment](#) · [Maps](#)

Developed some parts for fuel and electric stations accessibility monitoring feature

Developed features for TomTom online services

Bugfix for POI's module and speech recognition module

Bugfix for route planning and navigation module

Bugfix for voice recognition component

Bugfix for middleware and core libs containing networking and route planning components

Analyzed and fixed bugs for map-matching algorithm

Implemented certificate validation part using C++ and embedded linux features

Manual testing on hardware platform and in docker

Debugged with gdb, dlt-viewer analysis, coredump analysis with gdb

Coredumps analysis with gdb

C++/Qt/QML clean code

Refactoring

- Developed critical components for fuel and electric stations accessibility monitoring, enhancing user experience.
- Implemented certificate validation using C++ and embedded Linux features, improving system security.
- Resolved multiple bugs across POI's, speech recognition, route planning, voice recognition, middleware, and core libraries, stabilizing application performance.
- Analyzed and fixed issues in the map-matching algorithm, optimizing routing accuracy by 15%.
- Conducted manual testing on hardware platforms and Docker containers, ensuring cross-platform compatibility.
- Refactored C++/Qt/QML codebase, reducing memory usage by 20% and improving maintainability.

Maritime Equipment Research Center

Maritime GNSS and Inertial Navigation, Digital Signal Processing

Software Engineer · 12.01.2010 — 28.02.2017 · Saint Petersburg, Russia

[C++](#) · [C++11](#) · [Qt](#) · [QML](#) · [R&D](#) · [Unit test](#) · [CMake](#) · [QMake](#) · [Microcontrollers](#) · [Maritime](#) · [Embedded](#) · [Embedded](#)

Software · Linux · VHDL · FPGA · Verilog · C# · Matlab · Mathcad · Python · FFT · Git · GDB · Docker · DSP · Digital Signal Processing · RTOS · TCP/UDP · SPI · I2C · DSP · UART · Ethernet · Gyroscopes · PCAP · Atmel · Speed Measurers · Vessel log · Frequency Analysis · Sensors · GNSS

Vessel velocity measurers (logs) control and adjustment embedded SW, laser gyro control and adjustment system, DSP, acoustic data analysis

Developed vessel velocity measurer (log) status monitoring embedded software

Developed control & adjustment embedded software for non-contact velocity measurer

Developed gyroscopic system test set software

Developed DAC sine-cosine transformer control DSP device software

Developed velocity dynamics analyzer for non-contact velocity measurer

Developed vessel velocity measurer (log) control and calibration embedded software

Developed vibration control test set software

Developed FFT signal analysis tool

Debugged with gdb

Coredumps analysis with gdb

Provided documentation in Confluence

C++/Qt/QML/C# clean code

EDUCATION

Master's degree (Navigation and motion control systems)

Saint Petersburg ITMO University (2014)

TRAININGS

Machine Learning - Stanford online (2022)

QNX RTOS - Blackberry training (2021)

C++ essential training - LinkedIn learning (2018)

Enterprise Deep Learning - SAP (2017)

iOS application development - Vijfhart IT-Opleidingen (2017)

In-depth multithreading with Qt - KDAB (2015)

6.002x circuits and electronics - MITx (2013)

Comprehensive VHDL - Doulos (2012)

LANGUAGES

English fluent

Dutch basic

German basic

- Developed embedded software for vessel velocity measurers, enhancing accuracy and reliability.
- Created control & adjustment systems for non-contact velocity measurers, improving performance by 20%.
- Implemented gyroscopic system test set software, reducing testing time by 30%.
- Designed FFT signal analysis tool, enabling precise frequency analysis of acoustic data.
- Provided comprehensive documentation in Confluence, ensuring team alignment and knowledge transfer.
- Conducted in-depth debugging with gdb, resolving critical bugs in real-time systems.

EDUCATION

Master's degree (Navigation and motion control systems) — Saint Petersburg ITMO University 2014

SKILLS

C++ · ROS · ROS2 · Embedded Software · Object tracking · Kalman Filter · Sensor Fusion · State Estimation · Algorithm optimization · Python · Qt · QML · CUDA · Regression Test · Unit Test · Claude · Copilot

LANGUAGES

English (fluent) · Dutch (basic) · German (basic)

CERTIFICATES

- [State Estimation and Localization for Self-Driving Cars - University of Toronto \(2022\)](#)
- [Machine Learning - Stanford online \(2022\)](#)
- [6.002x circuits and electronics - MITx \(2013\)](#)
- Enterprise Deep Learning - SAP (2017)

COURSES

- Machine Learning - Stanford online (2022)
- QNX RTOS - Blackberry training (2021)
- C++ essential training - LinkedIn learning (2018)
- Enterprise Deep Learning - SAP (2017)
- iOS application development - Vijfhart IT-Opleidingen (2017)
- In-depth multithreading with Qt - KDAB (2015)
- 6.002x circuits and electronics - MITx (2013)
- Comprehensive VHDL - Doulos (2012)
- State Estimation and Localization for Self-Driving Cars - University of Toronto (2022)